Development of a Heterogeneous Reconfigurable CGRA Cell

Duarte Sanchez David, Jimenez Ulate Eddy, Ruiz Rivera Jeffrey, Esquivel Arias Fabian Zúñiga Ramírez Jose Eduardo
davidduarte28, e.jimenez.10, yr1ruiz, fabiesqui22, je.zuniga@estudiantec.cr

I. Introduction

Reconfigurable architectures have been very successful in industry because software evolution has advanced rapidly, with diverse applications, different user needs, and scientific progress, so hardware cannot adapt at the same pace as software [1]. Special-purpose systems can suffer from rapid obsolescence, and if the price of hardware development and production is high, it becomes economically unviable [1].

Currently, architectures such as FPGAs (*Field Programmable Gate Arrays*) have been widely adopted in diverse applications due to their flexibility and customization capability, gaining traction in High-Performance Computing (HPC). However, FPGAs present certain limitations such as slow configuration times, long compilation times, and, although somewhat relative, low operating frequencies [2].

Due to the above, there has been growing interest in HPC in using architectures such as CGRAs (*Coarse-Grained Reconfigurable Arrays*), which offer an intermediate solution between FPGAs and general-purpose processors (GPPs). CGRAs are hardware architectures that, conceptually, allow silicon to be malleable and their functionality to be dynamically reconfigured [2]. CGRAs have the advantage of being one to two orders of magnitude more efficient than FPGAs and more than 2 to 3 orders of magnitude more efficient than GPPs [1].

The development of a reconfigurable CGRA cell is of great importance not only for its relevance in HPC, but also for its adoption in *Machine-learning* applications, image processing, and computer vision, communications, and other areas [3]. Therefore, the objective of this work is to develop a heterogeneous CGRA, tentatively featuring data-routing cells, cells with distributed memory banks, scalar processing cells, and vector processing cells, capable of executing programs using the RISC-V instruction set, with the ability to execute 8, 16, and 32-bit logic.

The importance of using this instruction set is that it is an open standard, facilitating its adoption since it requires no license payments. This is an instruction set based on simplicity, making hardware design simpler and its implementation more direct [4].

II. Background and Related Work

In recent years, CGRAs have gained significant prominence as hardware accelerators due to their ability to offer an exceptional balance between software flexibility and the high performance of application-specific hardware [1], [2]. Unlike fine-grained architectures, CGRAs operate at the word or block level, which substantially reduces reconfiguration and routing overhead, making them particularly attractive for efficient edge-computing applications. The development and research of these architectures has been strongly driven by the adoption of the open-source RISC-V instruction set architecture (ISA) [4], whose extensible nature facilitates the creation of highly customized heterogeneous Systems on Chip (SoCs).

Recent research has demonstrated the advantages of tightly integrating RISC-V cores with CGRA arrays to overcome the traditional Von Neumann bottleneck. For example, [5] explore the integration of a CGRA accelerator coupled to a high-performance RISC-V CVA6 core focused on cloud-edge ecosystems, while [3] propose DVHetero, a comprehensive framework for the design and validation of heterogeneous SoCs based on this combination.

At the level of practical implementation, these architectures materialize in cutting-edge open-source efforts; repositories such as risc-v-cgra (developed by ECASLab) investigate these integrations at the system level, while projects such as OpenEdgeCGRA (ESL-EPFL) provide complete open-hardware platforms. In the latter, the architecture is composed of a configurable two-dimensional mesh of Processing Elements (PEs), or reconfigurable cells, that interact in a toroidal fashion and share registers to maximize instruction-level parallelism with minimal memory overhead. Another notable project is OpenCGRA, developed by PNNL (*Pacific Northwest National Laboratory*), which generates reconfigurable processors in a parameterizable way, allowing the hardware to be designed via code — including mesh size, cell type, and how cells communicate with each other — after which functionality is tested using Python and Verilator, and finally the design is exported to a standard hardware description code (HDL) such as Verilog.

For this type of system to reach its full potential, the internal design of the individual reconfigurable cell (PE) and its associated compilation tools are critical [6]. They highlight the need for context-memory-aware mapping to ensure energy-efficient acceleration, emphasizing that data-access latency dictates overall performance. Within this framework, the design and development of a new reconfigurable CGRA cell is grounded in the need to optimize local interconnects, deterministic access to intermediate operands, and the versatility of its cycle-level operations — challenges that remain areas of active exploration at the frontier of heterogeneous accelerator

design.

III. Analysis of Existing Solutions (SOTA)

Currently, there are different types of CGRAs, which can have an array of homogeneous PEs, as is the case with the HyCUBE and DySER CGRAs [7], or they can be heterogeneous, such as Plasticine, which has two types of units, PCUs and PMUs [8]. Another important characteristic to highlight is the type of precision these devices can support, where many of them are restricted to processing only integer operations, making them more general-purpose, while others do add the advantage of floating point for more specialized processes [9].

CGRA systems are no longer seen as an isolated element; today they are presented as a heterogeneous part of an architecture system [10]. Furthermore, CGRAs have become highly important in HPC (*High Performance Computing*) systems, which greatly benefit from having heterogeneous-type PEs, with a mix of basic and complex PEs [9]. This is highly important for mathematical functions, since traditional CGRAs do not handle them optimally.

Table I shows a comparison of three types of CGRA; the construction of these devices can vary, generating different applications and *trade-offs* where they are used according to the application.

Table I
Comparative matrix of state-of-the-art solutions

Solution	Strengths	Weaknesses	Viability
Proposal A [11]	Low latency, spatial optimization.	Hardware rigidity, high design time.	Fixed, stable domains.
Proposal B [12]	High flexibility, agile reconfiguration.	High resource consumption, *overhead*.	Non-critical dynamic environments.
Proposal C [8]	Massive per-cycle processing (OP/s).	Restrictive power consumption.	Fixed infrastructure/servers.

The design of these systems is no longer only necessary at the *hardware* level; it is also relevant to take their *software* counterpart into account [8]. Cases such as the one presented in [10] prioritize validation, mapping, and routing as problems just as important as the *hardware* side.

IV. Proposed Solution and Second-Level Architecture

A. General Description and Scope

This work presents a **heterogeneous reconfigurable cell** for CGRA [13], overcoming homogeneous architectures with under-utilization under heterogeneous workloads [11], [12]. It introduces **four specialized cell types** in a single *fabric*:

1) **Routing:** FIFOs for dynamic, non-blocking flow.
2) **Distributed memory:** Local SRAM banks (1–4 KB) eliminate bottlenecks.
3) **Scalar processing:** Multi-precision ALU (int8/16/32) + hardware loops.
4) **Vector processing:** float32 SIMD (4–8 lanes) for 4–8 simultaneous elements.

Table II maps SOTA challenges to solutions.

Table II
SOTA challenges and proposed solutions.

ID	Challenge	Solution
C1	Homogeneous PEs → under-utilization	4 specialized cells (routing, mem., scalar, vector)
C2	Routing overhead > 30% area	Dedicated routing cells with FIFOs
C3	Centralized memory → bottlenecks	Local distributed SRAM (1–4 KB)
C4	RISC-V config. latency	CSR + DMA; per-context partial config.
C5	Simultaneous multi-precision	Scalar ALU int8/16/32; float32 VALU
C9	NoC → bottleneck	2D mesh, 32 b links, credit-based FIFOs

1) Architectural Novelty: Unprecedented combination: (a) routing as a primary cell with FIFOs, (b) per-cell distributed memory, (c) integer-float multi-precision in differentiated cells, (d) native RISC-V coupling (RV32IMAF) via CSR + DMA. None of the reviewed architectures (ADRES, HyCUBE, Plasticine, etc.) integrate these 4 features simultaneously.

2) Scope and Deliverables:

- **RTL (VHDL):** 4 cells + basic NoC router.
- **Integration:** 2×2 array with mesh NoC.
- **Verification:** Exhaustive testbench + randomized programs [14].
- **FPGA synthesis:** Subject to availability (resources + $F_{\max}$).
- **Benchmarks:** Matrix multiplication, FFT, vector reduction.

Metrics in Table III.

Table III
Evaluation metrics (HLS + FPGA flow).

Metric	Unit	Tool
FPGA resources	LUT/FF/DSP/BRAM	Vitis HLS / Vivado 2023.1
$F_{\max}$	MHz	Vivado Timing
Latency	cycles	C/RTL Co-simulation (HLS)
Throughput	OP/s, GFlop/s	Analytical analysis ($F_{\max}$ + latency)
Power	mW	Vivado Power
SW speedup	×	SystemC vs. RISC-V (C++)

B. Second-Level Architecture: Components

2×2 mesh topology with NoC (credit-based control). Fig. 1 shows the diagram.

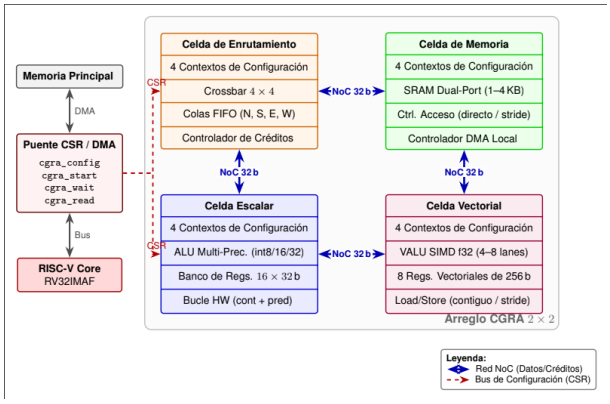

Figure 1. Second-level architecture diagram generated with artificial intelligence.

1) Routing Cell: First-class citizen (not static multiplexers): 4–8-entry FIFOs $\times$ 32 b per port; 4×4 crossbar (1 cycle, multicast); credit-based control; 4 contexts (1-cycle switch). Routing overhead $< 15\%$ area (vs. $\sim 30\%$ in HyCUBE [12]).

2) Distributed Memory Cell: 1–4 KB dual-port SRAM (FPGA BRAM); controller (direct/indirect/stride); NoC interface (1–2 cycles); local DMA (host-free block transfers). Eliminates centralized contention.

3) Scalar Cell: int8/16/32 ALU (4 int8 SIMD ops/cycle); 16 $\times$ 32 b registers; HW loops (counter + predication); 4 contexts; 32 b credit-based NoC interface.

4) Vector Cell: 4–8-lane float32 SIMD VALU (add, MAC, reduce); 8 vector registers (128/256 b); strided load/store; 4 contexts; 128 b NoC (burst mode). Will consume $\sim 3\times$ DSP, delivering $\sim 4\times$ float32 throughput [9].

5) NoC Router: 2D mesh, 32 b links; credits + 4-flit FIFO; round-robin arbitration; 1 cycle/hop; CSR/DMA host port (cell config., DMA transfers, result readout).

6) RISC-V Interface: Dedicated CSR (cell config., context, start/stop); 1-channel DMA; interrupts/flags; 4 custom instructions (cgra_config/start/wait/read). Reduces config. latency (challenge C4) [10], [15].

V. Experimental Plan and Methodology

A. Testing and Stimulus

To test an electronic design, a testbench is needed, in charge of driving a stimulus into the design and facilitating monitoring of what happens in the design under test (DUT). As has been done in other designs, and in the various CGRA development frameworks, it becomes necessary to have an algorithm that the architecture is capable of executing. For this, one typically starts from so-called kernels: representative algorithms, typically coded in C (for example, FFT and GEMM), annotated with compiler directives on their loops, whose data-flow structure can be mapped onto a CGRA's PE array. This is, in fact, the approach followed by Morpher within the OpenCGRA ecosystem: starting from the C kernel, it generates the data-flow graph (DFG) and the scratchpad memory layout, and produces the test vectors that feed the simulation/emulation stage, which are used to verify functional correctness and obtain area and power estimates [16]. The use of FFT as a reference kernel is also consistent with the CGRA literature oriented toward signal processing, where it recurrently appears alongside routines such as IDCT, FIR, and IIR [2].

It is worth noting that even in well-established frameworks such as OpenCGRA, the authors themselves point out that the simulator relies on testbenches manually written by the user and does not automate test-data generation or end-to-end verification [16]. This reinforces the relevance of proposing, from the outset, a structured and progressive testing flow such as the one proposed below.

While a valuable goal is to achieve end-to-end execution of an algorithm such as FFT or GEMM compiled from start to finish —from the C program to the configuration bitstream and the CGRA's input data—, given the time available for the project's development, the first objective targets algorithmically simpler tests. The following steps are proposed:

1) **A single PE executing $c = a+b$ (constant) or $c = a \times b$.** This validates loading of the configuration word (config word), reading of registers/inputs, the ALU operation, and writing of the output result. It is, literally, the architecture's "hello world."
2) **Chains of operations, in pipeline mode:** accumulated sum ($y = x_0+x_1+x_2+\ldots+x_n$) and successive-squaring powers ($x,\ x^2,\ x^4,\ x^8,\ \ldots$).
3) **Simple MAC ($acc \mathrel{+}= a \times b$):** this is the base block of almost every real kernel (FFT, GEMM, convolutions), so if this test works correctly, confidence is gained to scale toward more complex designs.

Once these basic tests are passed, progress can be made toward typical benchmarks from the CGRA literature, such as MachSuite and PolyBench [17], [18], which together with Wavelib and BEEBS make up the set of suites used by Morpher to evaluate its compilation flow [16]. These suites provide kernel libraries (including FFT and GEMM) that can be mapped onto the CGRA through a compilation flow similar to the one described above.

B. Monitoring

Since this project will be developed in SystemC, the aim is to leverage the language's own features to implement performance measurement. It is proposed to implement a monitor as an `SC_MODULE` component that observes, non-intrusively, DUT elements such as signals and internal counters. This separation between functional logic and measurement logic follows SystemC's own modular design principle, in which the monitor can be added to or removed from the testbench without modifying the module under analysis.

It is proposed that the monitor compute, at minimum, the following metrics:

- **Operations per second (OP/s):** ratio between the total number of operations executed by the DUT and the total execution time.
- **Average latency:** recording `sc_time_stamp()` at the start and end of each individual operation, averaged over the sampling window.
- **Utilization rate:** proportion of active cycles relative to the total number of cycles.

These metrics are printed to the console via `SC_REPORT_INFO`.

Additionally, once the design reaches a stage suitable for FPGA or ASIC synthesis, it will be possible to measure power, area, and *timing* metrics using the corresponding EDA tools. This applies both to the AMD Kria KV260 and the AMD Alveo U55C.

C. Architectural Exploration

Tied to design-space exploration, the plan is to generate variations of the CGRA's architectural design by modifying the number and type of PEs (*processing elements*). For example, changes in the distribution of PE types (more memory banks,

Table IV
COMPONENT SUMMARY.

Component	Function	Key parameters
Routing	Interconnect + buffering	FIFOs 4–8 $\times$ 32 b; 4×4 crossbar; 4 contexts
Memory	Distributed SRAM	1–4 KB dual-port; DMA; 1–2 cycles
Scalar	Multi-precision integer	int8/16/32; 16 reg.; HW loops; 4 contexts
Vector	float32 SIMD	4–8 lanes; 8 reg.; MAC; 4 contexts
NoC Router	Mesh transport	32 b; credits; 1 cycle/hop; round-robin
RISC-V	Host-accelerator	CSR + DMA + custom ISA; RV32IMAF

more fixed-point ALUs, more floating-point ALUs), as well as different $n \times n$ array dimensions, aimed at optimizing the execution of vector instructions.

This type of exploration is consistent with the diversity of CGRA architectures reported in the literature, where performance depends heavily on the composition and arrangement of the PEs [2].

D. Preliminary Results

This subsection reports what has actually been measured against the plan of Sec. V, distinguishing completed results from work still in progress, consistent with the "preliminary" scope of this milestone.

1) Functional Verification: Table V summarizes the regression suite: 19 SystemC testbenches covering every module from Table IV individually plus three integration levels (mesh, mesh wrapper/CSR-DMA, RISC-V-facing system), totaling 139 individual pass/fail assertions with **zero failures**. One additional testbench (MAC-cell sum-reduction) currently fails to *build* due to a missing standard-library header, a build-pipeline issue rather than a design defect; it is tracked as a known, easily-fixable gap rather than hidden from this count.

Table V
REGRESSION SUITE RESULTS (SYSTEMC TESTBENCHES).

Scope	TBs	Asserts	Result
Cell-level (scalar, vector, MAC, memory, routing)	12	79	12/12 PASS
Mesh integration (1×1, heterogeneous 2×2, smoke/complex)	4	51	4/4 PASS
CSR/DMA wrapper + sum-reduction dataflow	2	16	2/2 PASS
RISC-V-facing system integration	1	1	1/1 PASS

2) End-to-End Sum-Reduction Demonstration: The mesh's routing, memory, scalar, and vector cells jointly execute $total = seed + \sum_{i=0}^{6} v_i$, driven entirely through the RISC-V-facing CSR/DMA bridge (no direct cell access from the testbench). Across three rounds with distinct signs and magnitudes, all outputs match the expected totals exactly. Counting the fixed-latency instruction-load/relay chain in the source (routing $\rightarrow$ memory $\rightarrow$ routing $\rightarrow$ scalar $\rightarrow$ vector, excluding the bounded DMA-completion poll), the dataflow completes in 27 fixed cycles plus at most 20 polling cycles (two single-element DMA transfers, each bounded at 10 cycles), i.e. $\leq$47 cycles per round at the simulated clock period — an upper bound derived from the source, not yet from an instrumented cycle counter.

3) HLS Cross-Validation: The arithmetic kernel of the sum-reduction benchmark was re-implemented as a standalone Vitis HLS C function and validated in C-simulation (`csim`) against the same three golden rounds used in the SystemC testbench, with an exact match on all outputs. This confirms the arithmetic contract is synthesis-tool-portable before committing to a full `csynth`/`cosim` run, whose resource and timing figures are not yet available and are deferred to the final report.

4) RISC-V Baseline Characterization: To contextualize the CGRA's expected benefit, the same sum-reduction kernel is being characterized as a compiled RISC-V (RV64GC) binary under gem5's system-call-emulation mode, comparing in-order (Atomic, Timing, Minor) and out-of-order (O3) CPU models via cycle counts in `stats.txt`. At the time of writing, the gem5 build is in progress; cycle-count results will be reported in the final article alongside the CGRA and HLS figures above.

E. Development Tools and Environment

In order to ensure strict reproducibility of the design and simulation tests, the software tooling infrastructure and its respective versions are defined in Table VI.

Table VI
DEVELOPMENT TOOLS AND ENVIRONMENT VERSIONS

Tool / Software	Version	Purpose in the Experiment
SystemC	3.0.1	Modeling and simulation of the CGRA architecture.
Xilinx Vivado Design Suite	2025.2	Logic synthesis and *timing* analysis.
ModelSim / Questa Advanced	2025.1	Cycle-accurate RTL functional simulation.

AI TOOL USAGE DECLARATION

For this work, Artificial Intelligence was used to help structure the introduction and state of the art, as well as for a grammar and spelling review of the document.

REFERENCES

[1] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. H. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Computing Surveys*, vol. 52, pp. 1 – 39, 2019.

[2] K. S. Artur Podobas and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE ACCESS*, vol. 8, 2020.

[3] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "Dvhetero: A framework for designing and validating heterogeneous soc with risc-v processor and cgra," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, pp. 1 – 30, 2025. [Online]. Available: https://www.semanticscholar.org/paper/8d8821ac4c9ec42fc45e662ff65a0ccd6361297a

[4] EdgeIR Staff. (2024, jan) What is risc-v and why is it important? online. Accessed: 2026-06-11. [Online]. Available: https://www.edgeir.com//what-is-risc-v-and-why-is-it-important-20240109

[5] J. Granja, D. Vázquez, A. Rodríguez, and A. Otero, "Integration of a cgra accelerator with a cva6 risc-v core for the cloud-edge," null. [Online]. Available: https://www.semanticscholar.org/paper/b4dfe9170dd1bfa04c4eee19794ebe685b388fe0

[6] S. Das, K. J. M. Martin, and P. Coussy, "Context-memory aware mapping for energy efficient acceleration with cgras," *Design, Automation & Test in Europe Conference & Exhibition*, vol. null, pp. 336–341, 2019. [Online]. Available: https://www.semanticscholar.org/paper/b3c002d29c59700cc1827aff29a72b560c86346a

[7] M. Wijtvliet, H. Corporaal, and A. Kumar, *Blocks, Towards Energy-efficient, Coarse-grained Reconfigurable Architectures*. Springer Nature Switzerland AG, 2022.

[8] R. Prabhakar, Y. Zhang, D. Koeplinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, "Plasticine: A reconfigurable architecture for parallel patterns," in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, Toronto, ON, Canada, 2017.

[9] E. Del Sozzo, X. Wang, B. Adhi, C. Cortes, J. Anderson, and K. Sano, "Exploration of Trade-offs Between General-Purpose and Specialized Processing Elements in HPC-Oriented CGRA," in *Proc. IEEE Int. Parallel Distrib. Process. Symp. (IPDPS)*. IEEE, 2024.

[10] G. Zhu, L. Deng, K. Zhang, W. Fan, B. Jin, W. Cao, F. Zhang, X. Zhou, F. Zhang, and X. Yu, "DVHetero: A framework for designing and validating heterogeneous SoC with RISC-V processor and CGRA," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 3, 2025.

[11] Mei, Bingfeng and Vernalde, Serge and Verkest, Diederik and De Man, Hugo and Lauwereins, Rudy , "ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix," in , 2004, kU Leuven.

[12] Karunaratne, Manupa and Kulkarni, Aditi and Mitra, Tulika and Peh, Li-Shiuan , "HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect," in , 2017, nTU Singapore.

[13] J. Ruiz, E. Jimenez, D. Duarte, E. Zuñiga, and F. Ezquivel, "Cgra systemc platform," https://github.com/ezrzuniga/CGRA-reconfigurable-cell-development/tree/main, 2026, gitHub repository, accessed July 26, 2026.

[14] B. Egger, E. Song, H. Lee, and D. Shin, "Verification of coarse-grained reconfigurable arrays through random test programs," in *Proceedings of the 19th ACM SIGPLAN//SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES)*, Philadelphia, PA, USA, 2018.

[15] A. F. R. Ribeiro, "CGRA-RISC: Simulation Infrastructure for Coupling CGRA Accelerator to RISC-V Processor," Dissertation, Mestrado em Engenharia Inform'atica e Computaç ao, Faculdade de Engenharia da Universidade do Porto, Porto, Portugal, Jul. 2024, supervisors: Dr. Jo ao Paulo de Castro Canas Ferreira and Dr. Nuno Miguel Cardanha Paulino. Submitted July 24, 2024.

[16] R. Juneja, P. Dangi, T. K. Bandara, Z. Li, D. Wijerathne, L.-S. Peh, and T. Mitra, "Building an open cgra ecosystem for agile innovation," in *Proceedings of the IEEE//ACM International Conference on Computer-Aided Design (ICCAD)*, 2025, pp. 1–9. [Online]. Available: https://arxiv.org//abs//2508.19090

[17] L.-N. Pouchet, "PolyBench/C: The polyhedral benchmark suite," https://www.cs.colostate.edu/~pouchet/software/polybench/, 2012, accessed: 2026-06-18.

[18] B. Reagen, R. Adolf, Y. S. Shao, G.-Y. Wei, and D. Brooks, "MachSuite: Benchmarks for accelerator design and customized architectures," https://github.com//breagen//MachSuite, 2014, gitHub repository. Accessed: 2026-06-18.